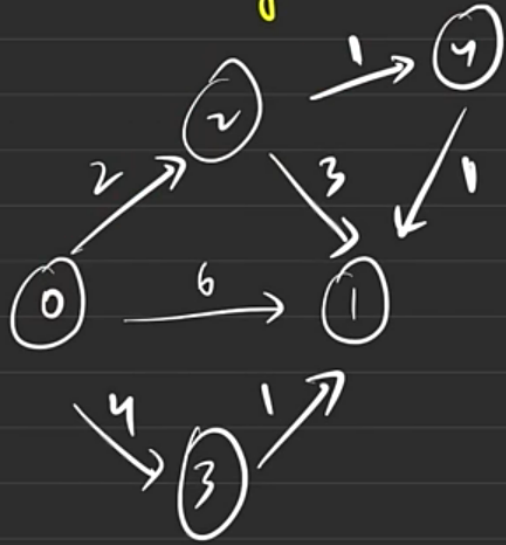


Floyd Warshall Algorithm $\rightarrow$ (multisource Shortest Path)
 $\rightarrow$ detect negative cycle



note $\rightarrow$ has via every vertex/node.

$$\begin{array}{cc} 0 & 1 \\ i & j \\ d[0][1] \rightarrow & \{0\} \rightarrow \{1\} \\ & (0 \rightarrow 1) \end{array}$$

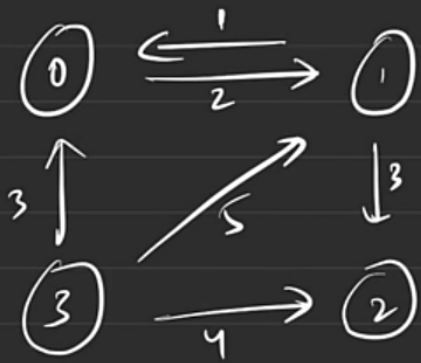
$$\binom{0}{2} + \binom{2}{3} = 5$$

$$(0 \rightarrow 3) + (3 \rightarrow 1) = 5$$

$$(0 \rightarrow \underset{3}{\underset{1}{\textcircled{4}}} + (4 \rightarrow 1) = 4$$

$$\min(d_{8;78K1} + d_{8;78K7})$$





	0	1	2	3
0	0	2	∞	∞
1	1	0	3	∞
2	∞	∞	0	∞
3	3	5	4	0

cost

	0	1	2	3
0	0	2	∞	∞
1	1	0	3	∞
2	∞		0	
3	3			0

$$\{1\} \{2\} \rightarrow \{1\} \{0\} + \{0\} \{2\}$$

1 + ∞

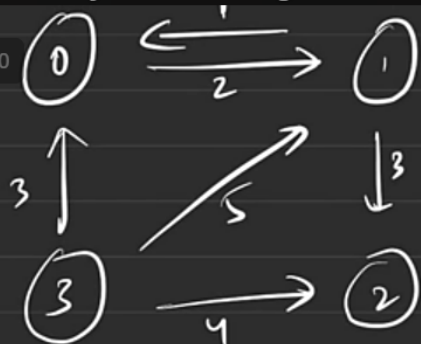
$$\{1\} \{3\} \rightarrow \{1\} \{0\} + \{0\} \{3\}$$

! + ∞



G-42. Floyd Warshall Algorithm

2.10



	0	1	2	3
0	0	2	5	5
1	2	0	3	5
2	5	3	0	4
3	5	5	4	0

cost

	0	1	2	3
0	0	2	5	5
1	2	0	3	5
2	5	3	0	4
3	5	5	4	0

$$\{0\} \{2\} \rightarrow \{0\} \{1\} + \{1\} \{2\}$$

2 + 3

$$\{0\} \{3\} \rightarrow \{0\} \{1\} + \{1\} \{3\}$$

2 + 5

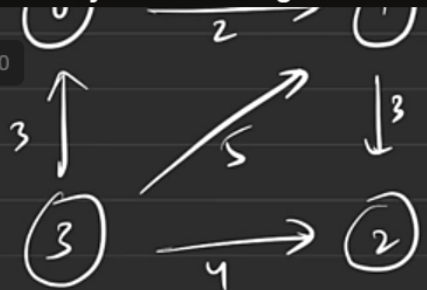
	0	1	2	3
0	0	2	5	5
1	2	0	3	5
2	5	3	0	4
3	5	5	4	0

via 1



G-42. Floyd Warshall Algorithm

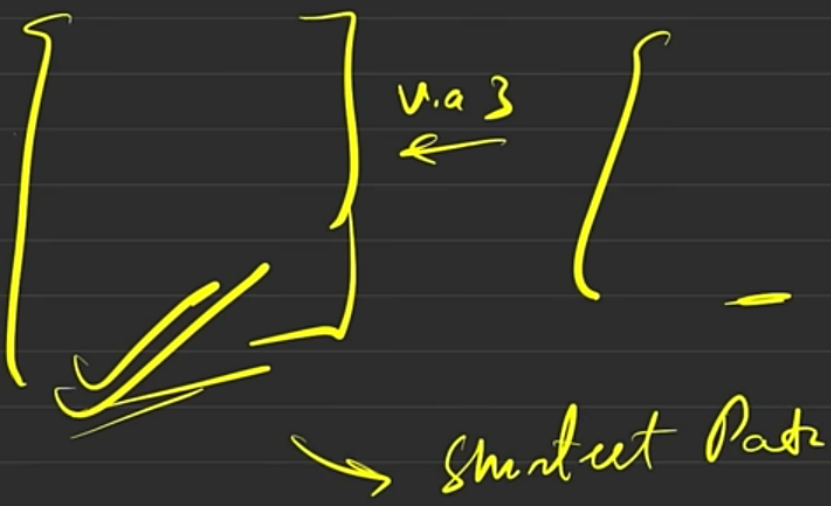
2.10



0	0	2	∞	∞
1	1	0	3	∞
2	∞	∞	0	∞
3	3	5	4	0

wst

0	0	2	∞	∞
1	1	0	3	∞
2	∞	∞	0	∞
3	3	5	4	0



via 2

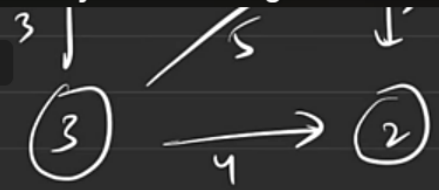
0	0	2	5	∞
1	1	0	3	∞
2	∞	∞	0	-
3	3	5	4	0

via 1



G-42. Floyd Warshall Algorithm

2.10



$$\begin{matrix} & 2 & 3 & 1 & 4 \\ \begin{matrix} 2 \\ 3 \end{matrix} & \begin{bmatrix} \infty & \infty & 0 & \infty \\ 3 & 5 & 4 & 0 \end{bmatrix}
 \end{matrix}$$

cost

$$\begin{matrix} & 2 & 3 & 1 & 4 \\ \begin{matrix} 2 \\ 3 \end{matrix} & \begin{bmatrix} \infty & \infty & 0 & \infty \\ 3 & 5 & 4 & 0 \end{bmatrix}
 \end{matrix}$$

via 1

```

for (via = 0; via < n; via++)
{
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {

```

$$\begin{matrix} & 0 & 1 & 2 & 3 \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 2 & 5 & \infty \\ \infty & 0 & 3 & \infty \\ \infty & \infty & 0 & - \\ 3 & 5 & 4 & 0 \end{bmatrix}
 \end{matrix}$$

$$\text{cost}[i][j] = \min \{ \text{cost}[i][j], \text{cost}[i][\text{via}] + \text{cost}[\text{via}][j] \}$$

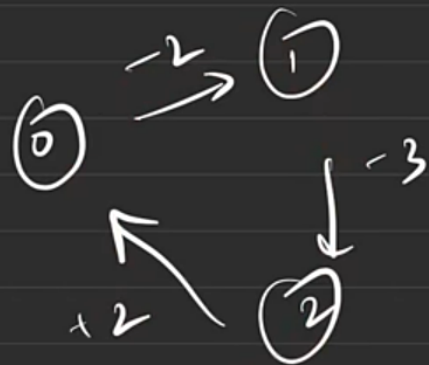
```

    }
}
}

```



→ How to detect a negative cycle



$$\begin{aligned} & -2 + 2 - 3 \\ \text{cost} &= \textcircled{-3} \end{aligned}$$

$\{$
 $\text{for } (i = 0 \rightarrow n)$
 $\{$

$\text{if } (\text{cost}[i][i] < 0)$
 $\{$

"negative cycle"

$\}$
 $\}$



G-42. Floyd Warshall Algorithm

Problems 2.10 Courses Get Hired Contests </>POTD

Practice

</> Problem

Editorial

Submissions

Comments

C++ (g++ 5.4)

Average Time: 15m

```
1 // } Driver Code Ends
2 //User function template for C++
3
4
5
6
7
8
9
10 class Solution {
11 public:
12 void shortest_distance(vector<vector<int>>&matrix){
13     int n = matrix.size();
14     for(int i = 0;i<n;i++){
15         for(int j = 0;j<n;j++){
16             if(matrix[i][j] == -1) {
17                 matrix[i][j] = 1e9;
18             }
19             if(i == j) matrix[i][j] = 0;
20         }
21     }
22
23     for(int k = 0;k<n;k++){
24         for(int i = 0;i<n;i++){
25             for(int j = 0;j<n;j++){
26                 matrix[i][j] = min(matrix[i][j],
27                                     matrix[i][k] + matrix[k][j]);
28             }
29         }
30     }
31
32     for(int i = 0;i<n;i++){
33         if(matrix[i][i] < 0) {
34             //
35         }
36     }
37
38
39
40     for(int i = 0;i<n;i++){
41         for(int j = 0;j<n;j++){
42             if(matrix[i][j] == 1e9) {
43                 matrix[i][j] = -1;
44             }
45         }
46     }
47 }
```

The problem is to find the shortest distances between every pair of vertices in a given edge-weighted directed Graph. The Graph is represented as an adjacency matrix, and the matrix denotes the weight of the edges (if it exists) else -1.

Do it in-place.

Example 1:

Input: matrix = {{0,25},{-1,0}}

Output: {{0,25},{-1,0}}

Explanation: The shortest distance between every pair is already given(if it exists).

Example 2:

Input: matrix = {{0,1,43},{1,0,6},{-1,-1,0}}

Output: {{0,1,7},{1,0,6},{-1,-1,0}}

Explanation: We can reach 3 from 1 as 1->2->3 and the cost will be 1+6=7 which is less than 43.

Your Task:

You don't need to read, return or print anything. Your task is to complete the function `shortest_distance()` which takes the

